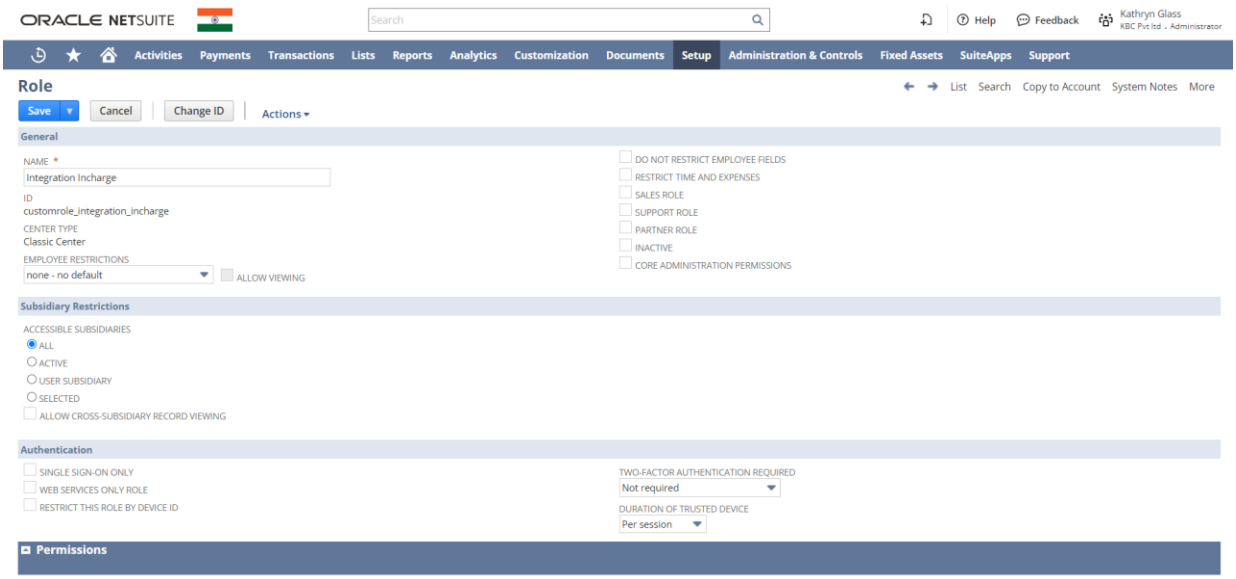
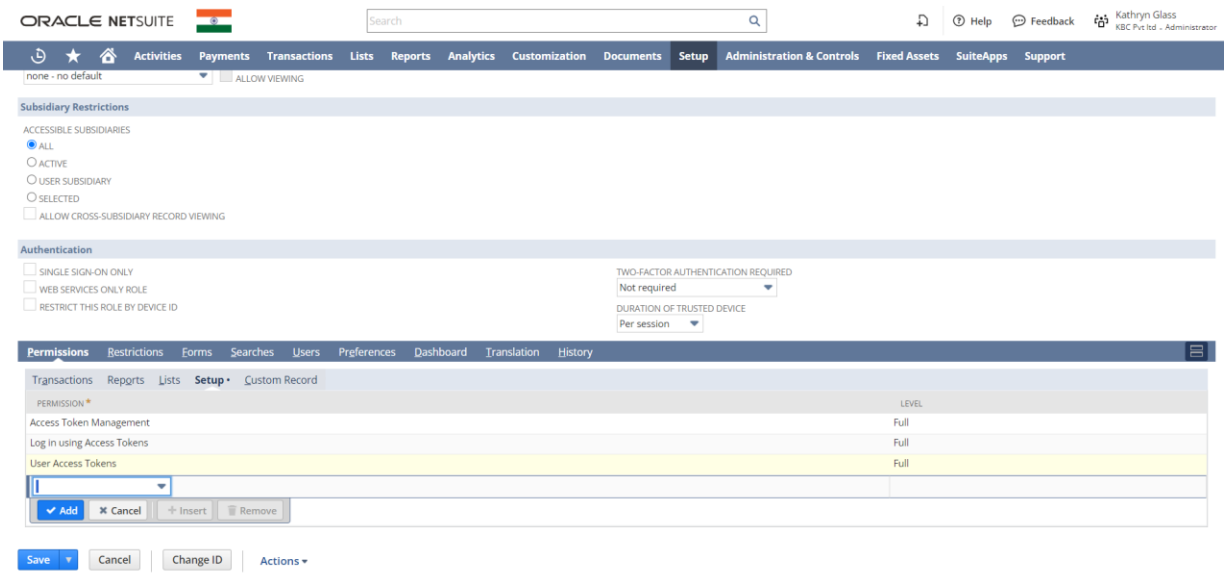
Create a Restlet :

1.Create a new role i.e. “Integration Incharge” : Set-up > User/Roles > Manage roles > New

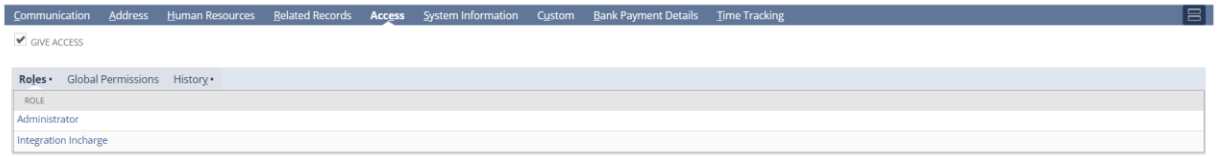
2.New role will have access level of -

- Access Token Management
- Log in using Access Tokens
- User Access Tokens

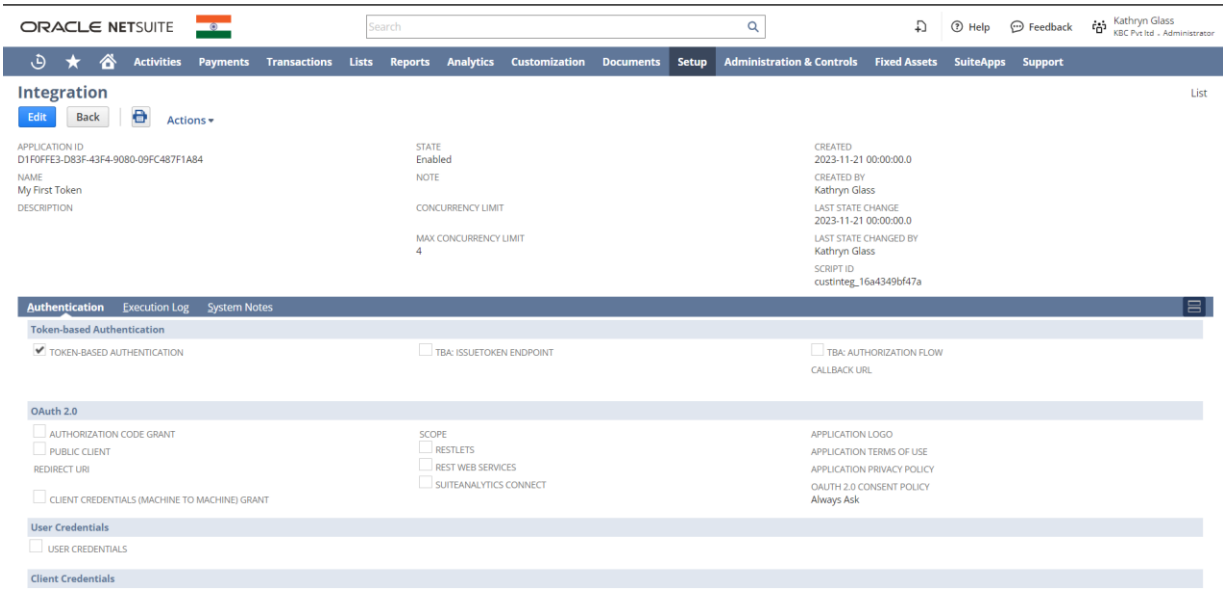


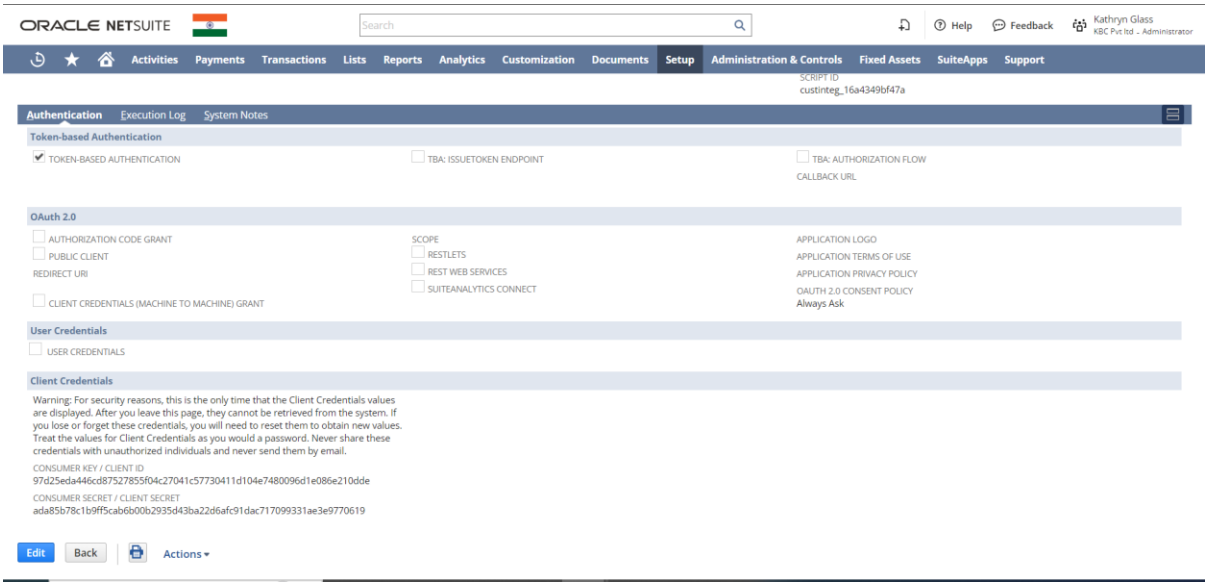


3.Assign role “Integration Incharge” to Kathryn Glass Employee: Relationship > Employee > Edit ‘Kathryn Glass’ > Access > Roles > Select ‘Integration Incharge’



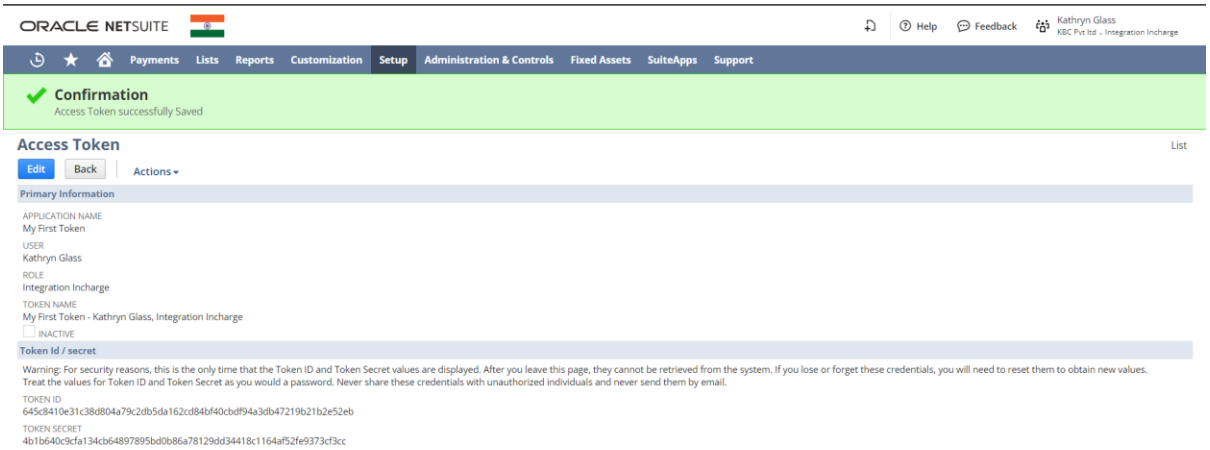
4.Create a new Integration: Set-up > Integration > Manage Integration > New

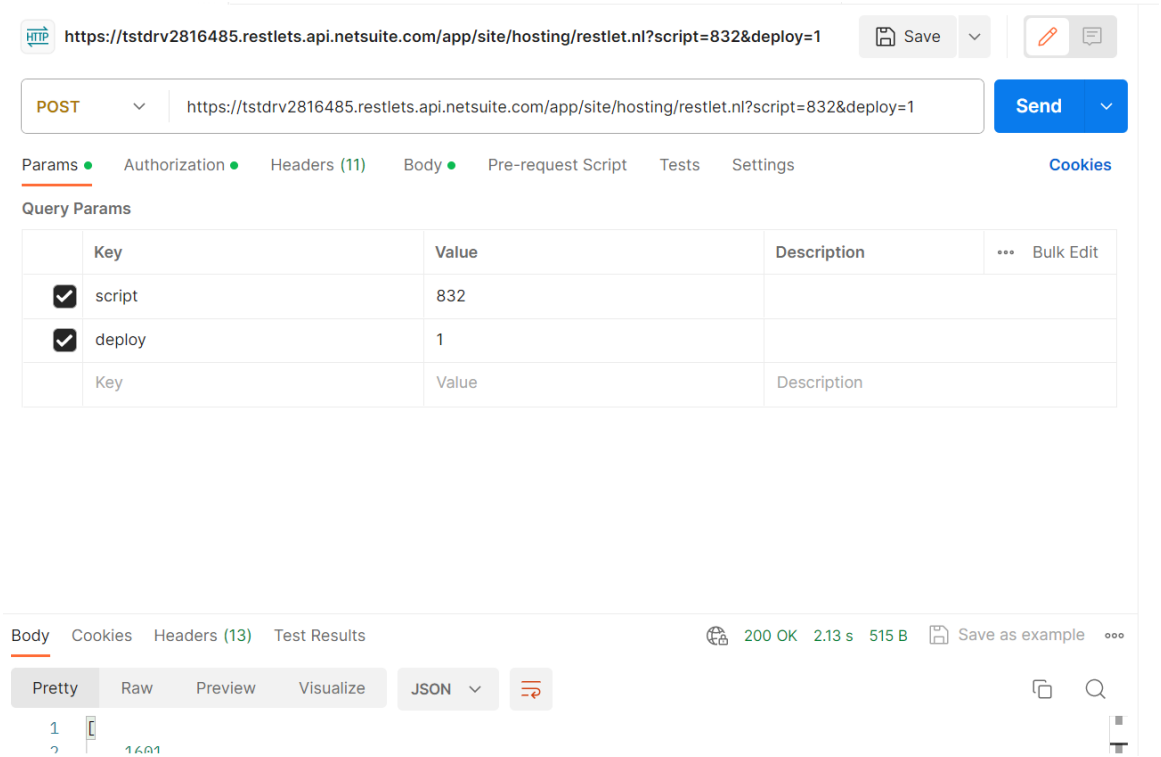


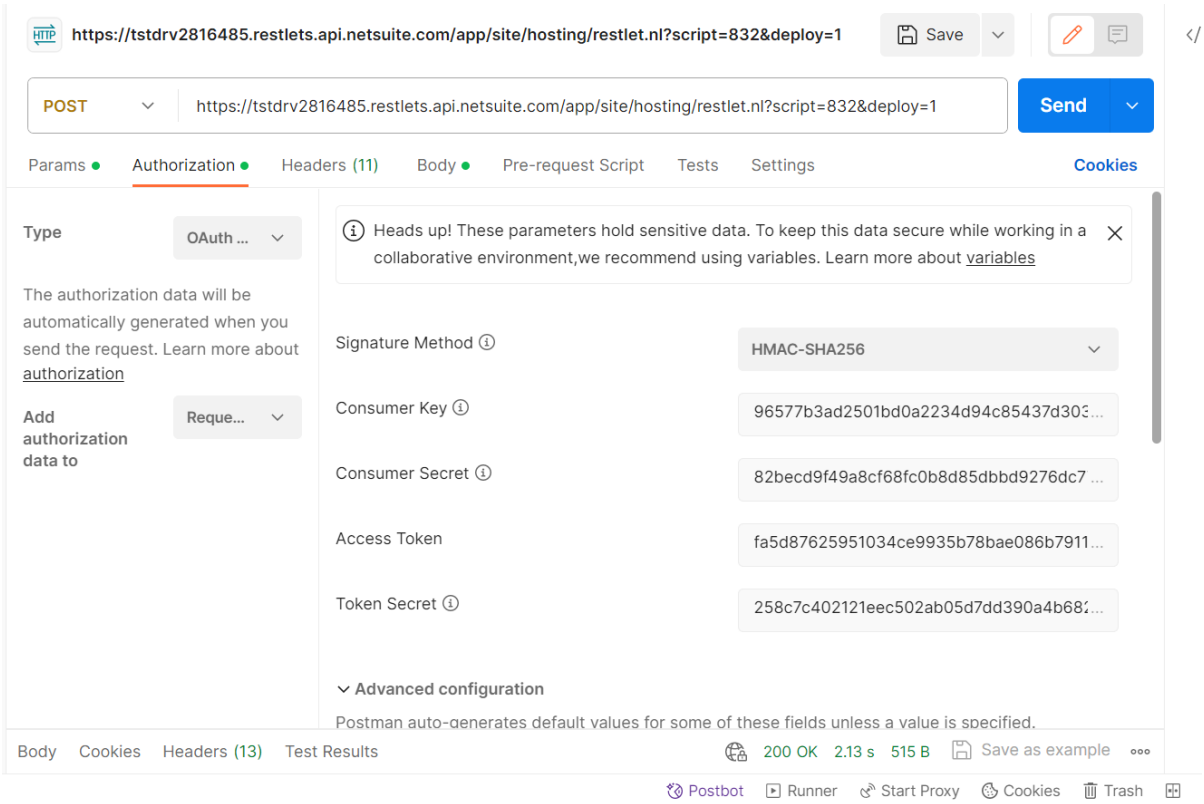


5.Login in NetSuite with “Integration Incharge” role

6.Create an access token: Set-up > User/Roles > Access Token > New and Set token in Postman







7.Again login with Administrator role, and create Restlet Script and deploy it

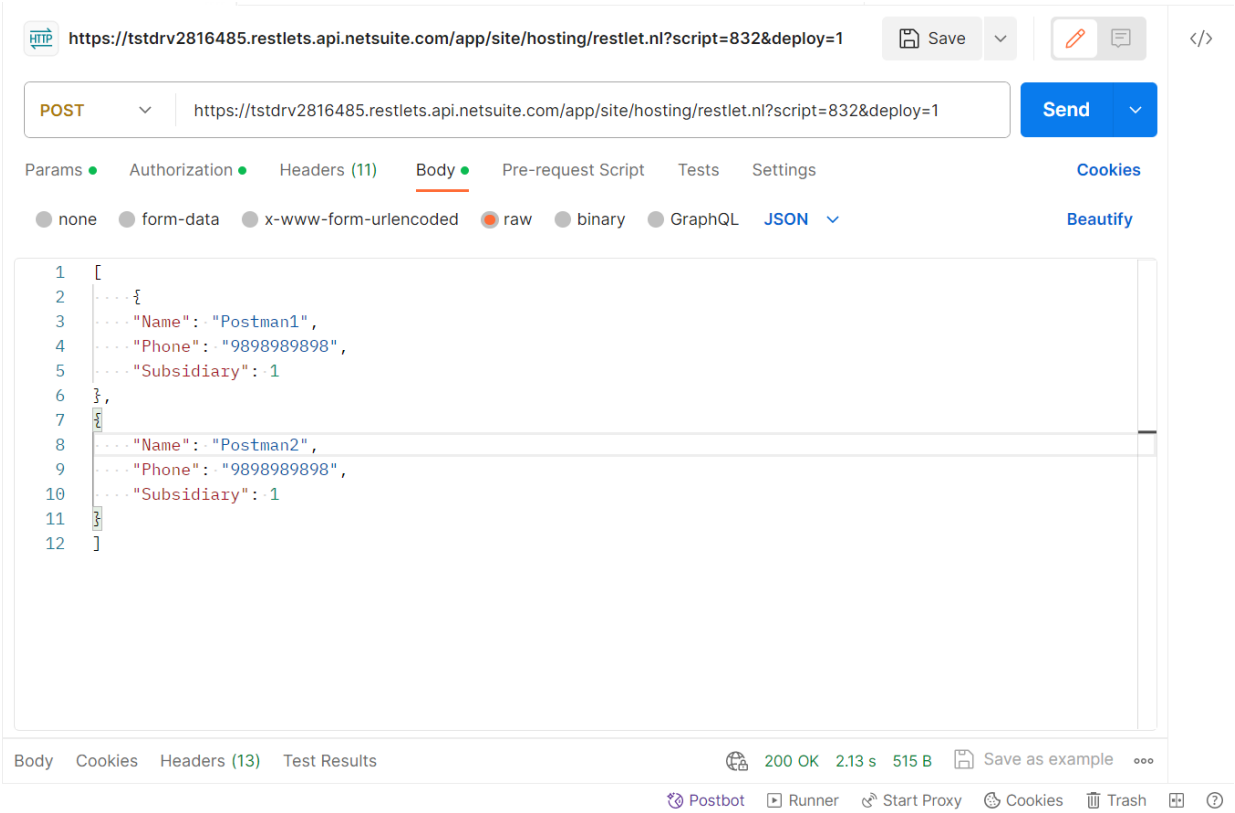
8.RestLet script to create two customer

```
/**
 * @NApiVersion 2.1
 * @NScriptType Restlet
 */
define(['N/currentRecord', 'N/record'],
    /**
 * @param{currentRecord} currentRecord
 */
    (currentRecord, record) => {

        const get = (requestParams) => {

            return 'Script Running';

        }

            const post = (requestBody) => {

                var arr = []

                for (var i = 0; i < requestBody.length; i++) {

                    var customerName = requestBody[i].Name
                    var customerPhone = requestBody[i].Phone
                    var customerSubsidary = requestBody[i].Subsidiary

                    var customerRecord = record.create({
                        type: record.Type.CUSTOMER,
                        isDynamic: true,
                    });
```

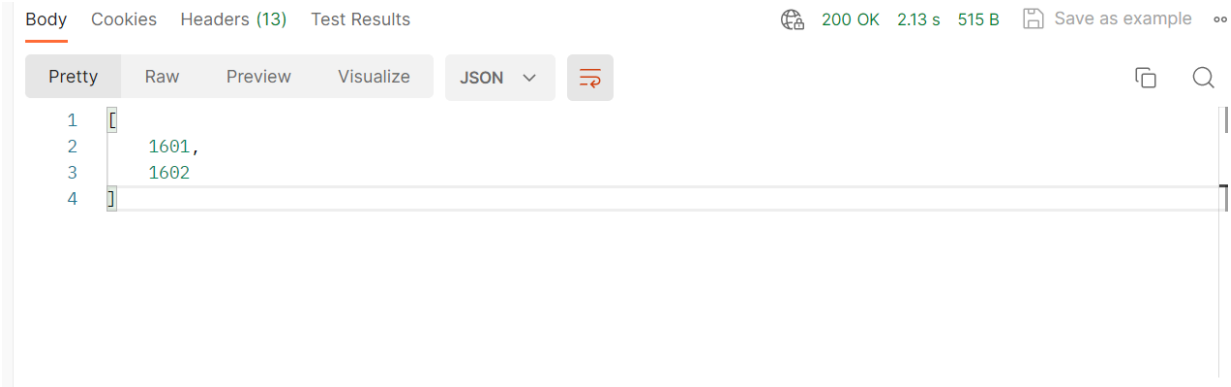
```
var arr = []

for (var i = 0; i < requestBody.length; i++) {

    var customerName = requestBody[i].Name
    var customerPhone = requestBody[i].Phone
    var customerSubsidary = requestBody[i].Subsidiary

    var customerRecord = record.create({
        type: record.Type.CUSTOMER,
        isDynamic: true,
    });

    customerRecord.setValue({
        fieldId: 'companyname',
        value: customerName,
    });
    customerRecord.setValue({
        fieldId: 'phone',
        value: customerPhone,
    });

    customerRecord.setValue({
        fieldId: 'subsidiary',
        value: customerSubsidary,
    });


    var customerId = customerRecord.save();
    log.debug('custId', customerId);
```

```
        fieldId: 'phone',
        value: customerPhone,
    });

    customerRecord.setValue({
        fieldId: 'subsidiary',
        value: customerSubsidary,
    });


    var customerId = customerRecord.save();
    log.debug('custId', customerId);

    arr.push(customerId)


}
return `Customer has been created and these are Ids : ${arr}`


}


const put = (requestBody) => {


}
```

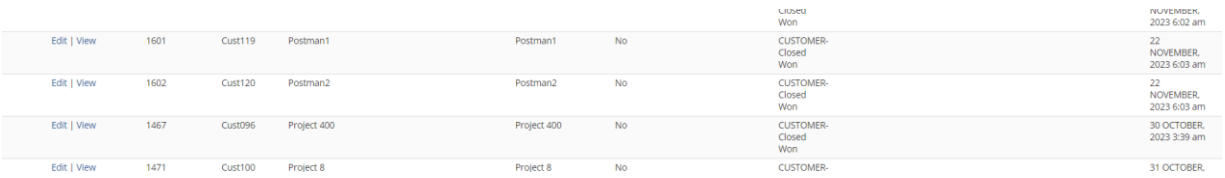
9.Postman trigger restLet



10.Id has been returned through script



11.Customer has been create



12.Use put method to update Email id of customer

```
const put = (requestBody) => {

    var arr = []

    for (var i = 0; i < requestBody.length; i++) {

        var CustomerId = requestBody[i].CustomerId;
        var Email = requestBody[i].Email;


        const customerRecord = record.load({
            type: record.Type.CUSTOMER,
            id: CustomerId,
            isDynamic: true
        });

        customerRecord.setValue({
            fieldId: "email",
            value: Email
        });

        customerRecord.save();
        log.debug("Email added to Customer ID", CustomerId);

        arr.push(`Email ${Email} added to Customer ID ${CustomerId}`)



        log.debug('arr', arr)

    }
    return 'Email Id has been set'



}
```

13.Delete method to delete same customer record

```
const doDelete = (requestParams) => {



    var CustomerId1 = requestParams.CustomerId1

    var CustomerId2 = requestParams.CustomerId2

    record.delete({
        type: record.Type.CUSTOMER,
        id: CustomerId1
    });
    log.debug("Customer deleted", CustomerId1);


    record.delete({
        type: record.Type.CUSTOMER,
        id: CustomerId2
    });

    log.debug("Customer deleted", CustomerId2);



    return 'Customer hass been delete'



}

return { get, put, post, delete: doDelete }

});
```